University
Mohammed VI
Polytechnic

College of
Computing

Deliverable 6: SQL – DDL and DML

Data Management Course

UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering

Session: Fall 2026

Team Information

Team Name	.....
Member 1	Yassine Taha
Member 2	Name
Member 3	Name
Member 4	Name
Repository Link	`https://github.com/...`

---

1 Introduction

This deliverable implements the final RLMS relational schema in MySQL 8.0. It covers the DDL script (`ddl.sql`), a DML script (`dml.sql`) that populates the database, applies the eight modification operations of Part 2 and runs the 24 queries of Part 3, and the answer to the open question of Part 4.

2 Requirements

- Part 1: create all tables with data types, primary keys (composite for `Maintenance` and `CalibrationRecord`), foreign keys, `NOT NULL`, `DEFAULT` and referential actions, using the revised People schema.
- Part 2: populate the database and perform the eight DML operations.
- Part 3: write queries A1–A5, B6–B11, C12–C13, D14–D19, E20–E23 and F24.
- Part 4: identify three requirements that SQL constraints seen so far cannot enforce.

3 Methodology

Team roles. Member 1 wrote the DDL script, Member 2 the population data and modifications, Member 3 the queries of Part 3, Member 4 the report and the testing.

Design choices.

- Subtypes share the key of their parent: `Student`, `Employee` reference `Person`; `Academic`, `Administrative`, `Technical` reference `Employee`; `Faculty` references `Academic`.
- `Maintenance` and `CalibrationRecord` use (`unitId`, sequence number) as primary key, `Stocks` uses (`labId`, `consId`), and `Consumes` references `Stocks(labId, consId)`.
- `Supplies` references `Laboratory` and `Consumable` separately, so a supplier can serve a consumable the laboratory does not stock.
- `Reservation.status` defaults to `'Pending'`; `approvedByPid` is nullable.

Referential actions. `CASCADE` for dependent rows (`Reserves`, `Consumes` from `Reservation`, `Holds`, `Requires`, `Maintenance`, subtypes), because they have no meaning without their parent. `SET NULL` for `approvedByPid` and `supervisorPid`, so deleting an approver or a supervisor keeps the reservation or the employee. `RESTRICT` for rows that must not disappear silently: equipment units still reserved, laboratories with a supervisor, stocks already consumed, persons who made reservations.

Assumptions. (1) `email` is `UNIQUE` and `serialNumber` is `UNIQUE`; the requirements do not impose this. (2) Every reservation belongs to a research project. (3) Each stock has exactly one monitoring technician. (4) The relative date in reservation 2 uses `NOW()` so it always falls in the next fourteen days. (5) Identifiers are plain integers chosen by hand.

4 Implementation & Results

The dataset has 11 persons of all subtypes, 3 laboratories in 2 buildings (`Riad5`, `Building B`), 4 projects, 3 budgets, 4 models (one without units), 8 units, 8 reservations covering the five statuses, and 5 stocks (one below its reorder threshold). Part 2 operation 7 uses `UPDATE ... WHERE projId IN (SELECT ...)`; operation 8 deletes the test supplier. Queries use `JOIN`, `GROUP BY/HAVING`, `INTERSECT` (Q20), `NOT IN` (Q21), `NOT EXISTS` (Q22), `>= ALL` (Q23), and a double `NOT EXISTS` for division (Q24). Executed outputs are in the log file.

5 Discussion

Difficulties. Ordering the `CREATE TABLE` statements so referenced tables exist first, the composite foreign key from `Consumes` to `Stocks`, and the division query.

Part 4: rules SQL cannot enforce so far.

1. **Approval (RE3).** A reservation must have an approver exactly when its status is `'Approved'`. A foreign key only checks that the approver exists. The database would have to compare `status` and `approvedByPid` on every insert or update of a reservation.
2. **Reservation and participation exclusions.** `Administrative` staff may not reserve equipment or participate in projects. A foreign key can only require membership in a table, not absence from one, so the database would have to check that the person is not in `Administrative` when the row is inserted into `Reservation` or `ParticipatesIn`.
3. **Certifications.** A person may reserve a model only if they hold every certification it requires. This compares `Reservation`, `Reserves`, `Requires` and `Holds` across several tables, and must be checked when a unit is added to a reservation.
4. **Stock quantity.** The quantity consumed must not exceed the quantity on hand, and `Stocks.quantity` must decrease accordingly. This needs to be checked when a row is inserted into `Consumes`, with a read and an update of another table.

6 Conclusion

The RLMS schema was created, populated, modified and queried successfully with the constructs from the course. Rules spanning several rows or tables remain for the next lab on triggers.